

Agentes heurísticos y modelos de lenguaje para PyCatan

Shiyi Cheng

26 de abril de 2026

1. Objetivo

El objetivo del trabajo es implementar y evaluar un agente para PyCatan, primero con heurísticas clásicas y después con apoyo de LLM en decisiones acotadas. La memoria resume el diseño, los benchmarks realizados y las limitaciones observadas.

2. Diseño del agente heurístico

El agente heurístico se ha construido de forma incremental, activando mejoras por bloques de decisión. Esta estructura permite medir qué parte del comportamiento aporta más rendimiento. Las decisiones principales cubiertas son:

- **Colocación inicial:** evaluación de nodos por producción esperada, diversidad de recursos y calidad de la carretera inicial.
- **Construcción:** priorización ordenada de ciudad, pueblo, carretera y carta de desarrollo, eligiendo el mejor destino legal en cada caso.
- **Comercio:** uso de intercambios con la banca cuando permiten completar recursos críticos para el objetivo actual.
- **Ladrón y cartas:** reglas simples para no desperdiciar acciones y mantener una política estable.

La heurística de nodos combina la probabilidad del dado con la variedad de recursos. En Catan, producir de forma regular y no depender de un único recurso suele ser más importante que maximizar una casilla aislada. Para la construcción, el agente sigue la siguiente prioridad: ciudad > pueblo > carretera > carta y, dentro de cada tipo, elige el mejor destino según la puntuación heurística del tablero. La carretera se valora por su capacidad de abrir nuevos nodos útiles.

Decisión	Qué mira	Cómo decide	Para qué sirve
Colocación inicial	Números de los terrenos, variedad de recursos, puertos y si el nodo está en costa.	Puntúa cada nodo por producción y variedad. La carretera inicial se elige pensando en futuras salidas.	Empezar con recursos variados y margen para expandirse.
Construcción	Recursos propios y construcciones legales en el tablero.	Si puede, construye en este orden: ciudad, pueblo, carretera y carta. Para cada opción escoge el destino con mejor puntuación.	Convertir recursos en puntos y no dejar turnos muertos.
Comercio con banca	Recursos que sobran, recursos que faltan y tasa de puerto o banca.	Solo cambia recursos si el intercambio acerca al objetivo actual sin gastar materiales reservados.	Completar antes ciudades, pueblos, carreteras o cartas.
Ladrón y cartas	Terrenos ocupados, rivales adyacentes, cartas disponibles y recursos que faltan.	Evita bloquearse a sí mismo, apunta a rivales bien colocados y pide materiales útiles.	Reducir acciones desperdiciadas y mantener una línea de juego coherente.

Tabla 1: Resumen de las decisiones heurísticas implementadas.

3. Infraestructura experimental

Los benchmarks comparten ahora un módulo común, `benchmark_common.py`, y dos wrappers para `vs_random` y `vs_standard`. La infraestructura genera CSV homogéneos con partidas, victorias, winrate, puntos medios, rango medio, decisiones LLM, fallback y proveedor/modelo cuando aplica. En la versión final del código, el benchmark también contabiliza `match_error_count` para no confundir excepciones de ejecución con derrotas normales al repetir experimentos.

Los experimentos heurísticos tienen semillas reproducibles. La evaluación iterativa trabaja con las semillas 42, 31415 y 2026, 25 partidas por posición contra random, es decir, 100 partidas por semilla. Los experimentos LLM combinan ejecuciones completas cuando el coste lo permite y ejecuciones ligeras para los modelos locales.

4. Resultados del agente heurístico

La Tabla 2 resume el experimento de ablación. La columna gate 70 % indica si la configuración supera el requisito mínimo frente a agentes random.

Iteración	Media winrate	Desv. típica	Mínimo	Gate 70 %
iter0 random baseline	0.2733	0.0047	0.2700	No
iter1 game start	0.2800	0.0294	0.2500	No
iter2 build phase	0.8433	0.0499	0.7900	Sí
iter3 commerce trade	1.0000	0.0000	1.0000	Sí
iter4 thief devcards	1.0000	0.0000	1.0000	Sí

Tabla 2: Resumen del pipeline iterativo frente a agentes random.

El salto más importante aparece al activar `on_build_phase`. La explicación está apoyada por la implementación: el simulador llama a la fase de construcción repetidamente hasta que el agente devuelve `None` o una construcción inválida. Por tanto, un agente que transforma recursos en construcciones válidas de forma sistemática aprovecha mucho mejor cada turno que un agente aleatorio. En `ShiyiBaseAgent`, la construcción sigue una prioridad fija ciudad > pueblo > carretera > carta y, para cada tipo, escoge un destino legal mediante la puntuación heurística del tablero. En cambio, el agente random puede no construir aunque tenga recursos o puede elegir una acción menos útil. Esta diferencia de política explica que la colocación inicial por sí sola apenas cambie el rendimiento, mientras que la fase de construcción eleva el winrate medio frente a random de 0.28 a 0.8433.

La fase de comercio termina de cerrar el comportamiento frente a random: al desbloquear intercambios con la banca orientados al objetivo actual, el agente alcanza 100 % de victorias en las semillas evaluadas. Esta afirmación también se corresponde con el código. El objetivo se calcula según las construcciones legales y los recursos que faltan; después, `choose_bank_trade` solo propone un intercambio si existe superávit suficiente para pagar la tasa de puerto o banca sin consumir los recursos reservados para la construcción objetivo. Es decir, el comercio no es aleatorio: intenta convertir excedentes en el recurso más escaso para ciudad, pueblo, carretera o carta.

La ablación sirve para ver cómo mejora el agente frente a random al añadir cada bloque de comportamiento. En cambio, no permite medir por separado cuánto aportan comercio, ladrón y cartas contra agentes estándar, porque esa comparación solo se ha hecho con el agente final completo. Por eso, frente a agentes estándar, el resultado relevante es el benchmark final de `ShiyiHeuristicAgent`.

La evaluación final compara el agente heurístico completo, ya con todas las mejoras activadas, frente a random y frente a agentes estándar. La Tabla 3 resume esos resultados. Cada fila contiene 100 partidas por seed.

Benchmark	Seeds	Partidas	Victorias	Winrate medio	Puntos medios
vs random	42, 2026, 31415	300	300	1.0000	10.1133
vs estándar	42, 2026, 31415	300	271	0.9033	9.7200

Tabla 3: Evaluación final de `ShiyiHeuristicAgent`.

Este resultado refuerza la conclusión experimental dentro de las semillas evaluadas: el agente no solo mejora con claridad sobre random, sino que mantiene un rendimiento alto frente a agentes estándar del simulador. La tasa de victoria debe interpretarse como la define el benchmark implementado: para cada partida se toma la traza final y se considera ganador al jugador con más puntos de victoria en ese momento. Normalmente la partida termina cuando algún jugador alcanza 10 puntos, pero si se llegara al máximo de rondas el resumen seguiría escogiendo al jugador con mayor puntuación final. En esta evaluación no interviene el agente LLM, así que estas victorias pertenecen al agente heurístico.

La explicación más prudente para una tasa tan alta frente a estándar no es que cada módulo individual haya sido demostrado como superior por separado, sino que la política final combina varias ventajas simples y consistentes: construye siempre que puede, prioriza ciudades y pueblos, escoge nodos productivos y variados, usa carreteras para abrir nuevas posiciones, y comercia con la banca solo cuando el intercambio acerca al objetivo actual. Además, las reglas de ladrón y cartas evitan algunas acciones desperdiciadas. Lo que queda como limitación es que el benchmark final no aísla cuánto aporta cada pieza contra agentes estándar, solo demuestra el rendimiento del conjunto completo.

5. Integración de LLM

El agente LLM parte de `ShiyiHeuristicAgent`. La integración consulta el modelo en `on_game_start` y `on_build_phase`. Cada llamada contiene el estado mínimo necesario, una lista indexada de acciones legales y una instrucción para devolver JSON con el campo `action_index`. Si la respuesta es inválida, fuera de rango o no llega a tiempo, el sistema registra fallback y toma la acción heurística.

Para controlar coste y longitud del prompt, el LLM no recibe todo el espacio combinatorio de acciones del juego, sino una lista corta de hasta 14 acciones legales ordenadas por una puntuación simple. Por tanto, el modelo actúa como selector sobre un subconjunto prometedor y legal, no como sustituto completo de la política heurística. Esta decisión reduce tokens, evita acciones inviables y facilita comparar modelos pequeños, aunque limita la capacidad del LLM para explorar jugadas fuera de esa lista.

El trabajo compara tres variantes de prompt: `compact_json`, `resource_focus` y `safe_legal`. Los experimentos finales se concentraron en `compact_json`, porque es el formato más corto y el que mejor encaja con el requisito de respuesta estructurada; además, la configuración final del agente trabaja con esa variante tanto en `on_game_start` como en `on_build_phase`. Durante las pruebas locales ha aparecido un fallo importante: algunos modelos pequeños tendían a copiar el JSON de entrada o a confundir `action_index` con `node_id`. Para corregirlo, el prompt pasó a incluir acciones explícitamente indexadas y la restricción de que `action_index` no es ni `node_id`, ni `road_to`, ni tipo de construcción.

Además del diseño, la memoria incluye una comparación ligera de prompts con Bedrock `nova-micro-v1`. La Tabla 4 muestra que los tres prompts consiguen ganar las partidas del ensayo, pero difieren en robustez de formato. En este caso, `safe_legal` fue el más estable porque no produjo fallback.

Prompt	Partidas	Winrate	Puntos medios	Fallback
<code>compact_json</code>	4	1.0000	10.25	0.0629
<code>resource_focus</code>	4	1.0000	10.00	0.0432
<code>safe_legal</code>	4	1.0000	10.00	0.0000

Tabla 4: Comparación ligera de prompts con Bedrock `nova-micro-v1`.

Esta prueba no pretende demostrar superioridad estratégica con solo cuatro partidas, pero sí sirve para evaluar el formato de interacción. En LLM aplicados a simuladores, un prompt que gana pero genera fallback es menos fiable que uno ligeramente más conservador y estable.

6. Resultados LLM

La Tabla 5 recoge los resultados principales para comparar modelos. La tabla los separa de los experimentos exploratorios porque no todas las corridas tienen el mismo tamaño ni la misma fiabilidad. El análisis marca como inválido cualquier resultado con fallback alto, porque en ese caso la política real no es el LLM, sino la heurística de respaldo.

Proveedor	Modelo	Benchmark	Prompt	Partidas	Winrate	Avg pts	Fallback	Uso
Bedrock	nova-lite-v1	random	compact	100	1.0000	10.04	0.0000	válido
Bedrock	nova-lite-v1	standard	compact	100	0.9100	9.75	0.0000	válido
Bedrock	llama3.3-70b	random	compact	100	1.0000	10.12	0.0002	válido
Bedrock	llama3.3-70b	standard	compact	100	0.9400	9.77	0.0002	válido
UPV	llama-mini	random	compact	100	1.0000	10.05	0.0143	aceptable
UPV	llama-mini	standard	compact	100	0.9000	9.63	0.0000	válido
Ollama	qwen2.5:0.5b	random	compact	100	0.8300	9.21	0.0000	válido parcial
Ollama	qwen2.5:0.5b	standard	compact	4	0.2500	4.75	0.0000	prueba corta limpia

Tabla 5: Resultados LLM principales.

Además de los resultados principales, el trabajo conserva experimentos exploratorios y descartes. La Tabla 6 resume todas las familias de experimentos realizados. En los casos con varias corridas, la tabla indica la ejecución

final o el patrón observado; las corridas de solo 4 partidas cuentan como pruebas cortas, no como benchmarks concluyentes.

Proveedor/modelo	Runs	Tamaño	Resultado observado	Fallback	Uso en el análisis
Bedrock nova-micro	20260421_225325, fase prompts	4 random, 8 standard; prompts de 4 partidas	Gana los ensayos ligeros	0.0256–0.0629	Exploratorio de formato y prompts
Bedrock Mistral 7B	20260422_091024, 20260422_093854	4+4 y 100+100	1.00 random, 0.89 standard en final	1.0000	Descartado: decide casi siempre el respaldo
Bedrock nova-lite	20260422_091024, 20260422_093854	4+4 y 100+100	1.00 random, 0.91 standard	0.0000	Válido
Bedrock llama3.3-70b	20260422_091024, 20260422_093854	4+4 y 100+100	1.00 random, 0.94 standard	0.0002	Válido, mejor Bedrock
UPV gpt-4o-mini	20260422_091742	4+4	Gana el ensayo	1.0000	Descartado por fallback total
UPV poligpt	20260422_111137	4+4	Gana el ensayo	0.3140/0.8625	Descartado por fallback alto
UPV llama-mini	20260422_111137, 113434, 120019	4+4, 4+4 y 100+100	1.00 random, 0.90 standard en final	0.0143/0.0000 final	Válido
UPV phi4-mini	20260422_113211, 120019	4+4 y 100+100	1.00 random, 0.87 standard en final	0.4288/0.5751 final	No fiable por fallback alto
UPV qwen3:4b	20260422_113726	4+4	Gana el ensayo	1.0000	Descartado por fallback total
Ollama qwen2.5:0.5b	20260424_092944, 121936, 123239, 123948, 124431	16 random; 4+4; 4 random; 4+4; 100 random	0.83 random limpio final; 0.25 standard en prueba corta limpia	de 0.7405 a 0.0000	Válido parcial; muestra mejora del prompt y límite de calidad local
Ollama gemma2:9b	20260424_101820, 122342	100 random; intento posterior incompleto	1.00 random en prueba antigua	0.9745	Descartado; no hay benchmark limpio posterior

Tabla 6: Experimentos LLM realizados, incluyendo corridas exploratorias y descartadas.

Estos descartes son importantes: si se ignora el fallback, un resumen puede mostrar victorias que en la práctica pertenecen al agente heurístico de respaldo. Por ejemplo, `mistral.mistral-7b-instruct-v0:2`, `UPV gpt-4o-mini`, `UPV qwen3:4b` y la ejecución antigua de Ollama `gemma2:9b` ganan o puntúan bien, pero sus decisiones no pueden atribuirse al modelo porque el respaldo domina la partida.

7. Análisis de modelos

Los modelos de Bedrock fueron los más estables. `nova-lite-v1` ofreció buen rendimiento sin fallback y `llama3.3-70b` obtuvo el mejor resultado frente a agentes estándar. `UPV llama-mini` también fue viable, aunque con un pequeño fallback contra random. La parte local incluye dos modelos mediante Ollama: `qwen2.5:0.5b` y `gemma2:9b`. Las primeras ejecuciones de `qwen2.5:0.5b` tuvieron mucho fallback o fueron pruebas cortas; tras endurecer el prompt y activar abortos por fallback, quedó completa una ejecución limpia de 100 partidas contra random. El resultado fue peor que Bedrock: 0.83 de winrate y 9.21 puntos medios. Esto sugiere que responder JSON válido no garantiza buena política.

La latencia local de `qwen2.5:0.5b` fue relativamente baja para un LLM: 488 ms medios por decisión en la ejecución completa contra random, con 8847 decisiones. Aun así, el coste total fue elevado porque el código llama al modelo muchas veces durante la partida. Por ello, una integración práctica debería reservar los LLM para decisiones de alto impacto o para generar heurísticas, no necesariamente para cada paso frecuente.

En cuanto a tokens y coste, la instrumentación actual del motor registra `input_tokens`, `output_tokens`, latencia y fallback cuando el proveedor los devuelve. La ejecución local completa con `qwen2.5:0.5b` promedió 356.64 tokens de entrada, 8.05 de salida y 488 ms por decisión en 8847 decisiones. En Bedrock y UPV, las ejecuciones comparables de la tabla principal salieron de una versión anterior del resumen experimental que no persistía latencia ni tokens; por ese motivo, esta memoria no compara numéricamente esos campos para dichos proveedores. Sí compara calidad de decisión, robustez de formato y facilidad de integración. Bedrock fue el más estable en formato y calidad, UPV resultó fácil de integrar con API tipo chat, y Ollama fue el más barato y controlable, aunque dependiente del hardware local y de modelos más débiles. Frente a agentes estándar, en local solo quedaron completadas pruebas cortas de 4 partidas para `qwen2.5:0.5b`; por tanto, esos datos quedan como indicios cualitativos y no como benchmarks comparables a Bedrock o UPV.

8. Viabilidad de los LLM en PyCatan

Los LLM pueden integrarse técnicamente en PyCatan, pero presentan tres riesgos. Primero, la salida estructurada no está garantizada: incluso con temperatura baja, algunos modelos devuelven índices fuera de rango o texto adicional. Segundo, la latencia multiplica el tiempo de benchmark. Tercero, si se permite fallback silencioso, se sobreestima la calidad del modelo.

La conclusión experimental es que los LLM grandes y bien servidos funcionan como política aproximada para decisiones acotadas, especialmente cuando reciben una lista de acciones legales. Sin embargo, para un agente competitivo, la heurística sigue siendo más barata, reproducible y fácil de depurar. El uso más razonable del LLM sería como apoyo en decisiones estratégicas poco frecuentes, generación de explicaciones o exploración de variantes heurísticas, no como sustituto completo de la política del agente.

Las principales limitaciones de este estudio son tres. Primero, el LLM opera sobre una lista corta heurística y no sobre todo el espacio de acciones, de modo que la comparación mide su capacidad de seleccionar entre opciones prometedoras. Segundo, no todos los proveedores registraron latencia y tokens con el mismo nivel de detalle en todas las corridas. Tercero, por coste computacional, la evaluación local frente a agentes estándar quedó limitada a pruebas ligeras.

9. Uso de herramientas de IA

Durante el desarrollo, los asistentes de programación basados en IA apoyaron la reorganización de benchmarks, el diseño del pipeline experimental, la implementación del motor LLM, la depuración de errores de formato JSON y la redacción de esta memoria. En concreto, ayudaron a extraer lógica repetida a `benchmark_common.py`, construir los prompts con salida JSON, revisar por qué algunos modelos confundían `action_index` con identificadores del tablero y detectar resultados contaminados por fallback. La limitación más clara apareció precisamente en la experimentación LLM: una salida aparentemente correcta puede esconder que la decisión real fue tomada por el respaldo heurístico, por lo que fue necesario validar con métricas, revisión del comportamiento y pruebas manuales.

10. Cobertura de criterios

La Tabla 7 resume cómo se cubren los puntos principales de la rúbrica.

Criterio	Evidencia incluida
Agente heurístico	Descripción de decisiones, información usada, reglas de construcción, comercio, ladrón y cartas.
Benchmarks	Ablación frente a random, evaluación final frente a random y estándar, semillas y comandos reproducibles.
Uso de LLM	Integración en <code>on_game_start</code> y <code>on_build_phase</code> , lista corta legal y respaldo heurístico.
Prompts y modelos	Tres prompts evaluados y modelos de Bedrock, UPV y Ollama, incluyendo resultados descartados.
Análisis experimental	Comparación por winrate, puntos, fallback, latencia/tokens cuando están disponibles y limitaciones de comparabilidad.
Herramientas IA	Uso explicado para refactorización, prompts, depuración y validación crítica de resultados.

Tabla 7: Relación entre la memoria y los criterios de evaluación.

11. Conclusiones

1. El agente heurístico supera ampliamente el objetivo de 70 % frente a random desde la iteración de construcción, y alcanza 100 % al añadir comercio en las semillas evaluadas.
2. La versión final obtiene 100 % frente a random y 90.33 % frente a agentes estándar en 300 partidas por benchmark.
3. La ablación frente a random muestra que `on_build_phase` es la mejora más decisiva; frente a agentes estándar, el benchmark final demuestra el rendimiento del conjunto completo, no la contribución aislada de cada módulo.
4. La integración LLM funciona en dos decisiones reales del agente: colocación inicial y construcción.
5. Se compararon tres prompts. `safe_legal` fue el más robusto en el ensayo ligero porque consiguió `fallback=0`.
6. Bedrock `nova-lite-v1` y `llama3.3-70b` son las configuraciones más sólidas entre las evaluadas.
7. Ollama local es viable técnicamente, pero los experimentos muestran mucha variación: algunas corridas antiguas tuvieron fallback alto y la ejecución limpia completa de `qwen2.5:0.5b` rindió peor que Bedrock.
8. El fallback debe tratarse como métrica crítica. Los resultados con fallback alto no son válidos para comparar modelos.